

High-Level Design (HLD) & Codebase Architecture Audit

Comprehensive Architectural Due Diligence, End-to-End Feature Tracing, Technical Debt Audit & Production Readiness Roadmap

Executive Summary: Astalakshimi is an enterprise South Indian matrimony platform engineered with a modern TypeScript monorepo architecture (pnpm workspaces + Turborepo). The system comprises a Next.js 15 App Router web frontend with a Backend-for-Frontend (BFF) HTTP-only cookie proxy, a modular NestJS 10 REST API backend, a PostgreSQL RDS database managed via Drizzle ORM (18 relational schemas), and dual-tier AWS S3 storage (public media and private verification vault). This High-Level Design document represents an exhaustive, evidence-based architectural audit of the active codebase, mapping every concrete feature, request lifecycle, dataflow, API endpoint, entity relationship, security vulnerability, and technical debt item.

Target Application: Astalakshimi Platform	Monorepo Root: /Users/karikalanloganathan/Desktop/Astalakshimi
Architecture Pattern: Monorepo (Next.js + NestJS + Drizzle)	Database: PostgreSQL 16 (AWS RDS) via Drizzle ORM
Frontend Framework: Next.js 15 (App Router, Tailwind, TanStack Query)	Backend API: NestJS 10 (TypeScript, Passport JWT, Zod)
Object Storage: AWS S3 (Dual-Bucket: Media + Vault) + CloudFront	Payment Gateway: Razorpay (Orders & HMAC Signature Verification)
Document Version: 2.0 (Deep Codebase Audit)	Audit Date: August 2026
Audited By: Senior Principal Software Architect & Codebase Auditor	Classification: Confidential / Engineering Blueprint

Table of Contents Overview:

1. Technology Stack Inventory & Monorepo Map	7. Authentication, Authorization & Security Audit
2. System Architecture & Request Lifecycle	8. Frontend State & Component Architecture
3. Master Feature Matrix & Domain Inventory	9. Backend Modular Architecture & Service Design
4. Deep Feature-by-Feature Technical Workflows	10. Redundant, Dead & Duplicate Code Audit
5. Master API Inventory Table (35+ Endpoints)	11. Architecture Gap Analysis (Specs vs Reality)
6. Database Architecture & ER Schema Specifications	12. Bottleneck Analysis & Actionable Roadmap

1. Technology Stack Inventory & Monorepo Topography

Astalakshimi is structured as a pnpm multi-package workspace managed by Turborepo. The codebase is divided into two runnable application packages (``apps/``) and five shared internal packages (``packages/``), supported by infrastructure and specification files.

Layer	Technology	Active Path / Package	Architectural Role & Scope
Monorepo	pnpm workspaces + Turborepo	Root: package.json, turbo.json	Workspace orchestration, build caching, dependency deduplication.
Frontend App	Next.js 15 (App Router), React 19, TypeScript	apps/web (<code>`@astalakshimi/web`</code>)	UI rendering, route groups, server-side data fetching, BFF cookie proxy.
Styling & UI	Tailwind CSS v4, Radix UI, Framer Motion	apps/web/src/components/ui	Accessible UI primitives, South Indian royal theme tokens, micro-animations.
Client State	TanStack Query (React Query v5)	apps/web/src/hooks/queries.ts	Server state synchronization, query caching, optimistic UI updates.
Backend API	NestJS 10, Express, Passport JWT	apps/api (<code>`@astalakshimi/api`</code>)	Core domain logic, REST controllers, guards, filters, transaction boundaries.
Database ORM	Drizzle ORM + postgres-js	packages/database (<code>`@astalakshimi/database`</code>)	18 SQL schemas, Drizzle Kit migrations, type-safe query builder.
Database	PostgreSQL 16 (AWS RDS)	Infrastructure / AWS Cloud	Relational persistence, unique constraints, foreign keys, btree indexes.
Storage & CDN	AWS S3 (Dual-Bucket) + CloudFront	apps/api/src/media/providers/s3.provider.ts	Public photos/horoscopes in Media bucket; secure IDs in Vault bucket.
Validation	Zod TypeScript Schema Validation	packages/validation (<code>`@astalakshimi/validation`</code>)	Shared contract validation for web forms and NestJS DTO pipes.
Shared Types	TypeScript Contract Definitions	packages/types (<code>`@astalakshimi/types`</code>)	Shared entity interfaces, auth payloads, upload contracts, profile DTOs.
Payments	Razorpay Node SDK	apps/api/src/payments	Order generation, payment capture, webhook event handling, signature checks.

Repository Topography & Package Status:

Directory / Package	Type	Active Status	Audit Observation & Recommendation
<code>`apps/web`</code>	Next.js App	Actively Used	Full App Router frontend. Handles BFF authentication, proxy, profile editing, and UI dashboard.
<code>`apps/api`</code>	NestJS API	Actively Used	Backend API service containing 18 domain modules, controllers, and services.
<code>`packages/database`</code>	Drizzle ORM	Actively Used	Source of truth for 18 database schemas, client provider, and SQL migrations.
<code>`packages/types`</code>	Shared Types	Actively Used	Shared TypeScript interfaces for Profile, User, Media, Preferences, Auth.
<code>`packages/validation`</code>	Shared Zod	Actively Used	Shared Zod validation schemas for forms and NestJS ZodValidationPipe.
<code>`packages/config`</code>	Shared Config	Empty Stub	Contains only <code>`.gitkeep`</code> . Constants are currently scattered across web and api. Safe to populate or consolidate.
<code>`packages/api-client`</code>	Shared HTTP Client	Empty Stub	Contains only <code>`.gitkeep`</code> . HTTP client was directly embedded into <code>apps/web/src/lib/api-client.ts`</code> .
<code>`infrastructure/`</code>	Docker Config	Empty Stub	<code>`docker/`</code> and <code>`docker-compose/`</code> contain only <code>`.gitkeep`</code> . Docker setup not yet configured.
<code>`scripts/`</code>	Build Scripts	Empty Stub	Contained <code>`.gitkeep`</code> . Scratch scripts (<code>`test-db.js`</code> , <code>`apps/web/fix_ts.js`</code>) were left scattered in root.

2. System Architecture & Request Lifecycle

Astalakshimi implements a decoupled 3-tier architecture with a secure Backend-for-Frontend (BFF) proxy pattern. The browser never directly receives or stores the JWT access token in JavaScript localStorage; instead, the Next.js API route attaches the token via HTTP-only, Secure, SameSite cookies.

CLIENT LAYER Next.js 15 Web App (React 19 / TanStack Query)	BFF PROXY LAYER Next.js Route Handler `app/api/proxy/[...path]`	DOMAIN API LAYER NestJS 10 REST API (`apps/api/src`)	PERSISTENCE & CLOUD PostgreSQL 16 (RDS) AWS S3 (Media/Vault)
• User Action (Form/Click) • React Hook Form (Zod) • TanStack Mutation / Query • `apiClient` fetch call	• Reads HTTP-only cookie • Injects `Authorization: Bearer` • Auto-refreshes 401 tokens • Strips sensitive headers	• Global Exception Filter • JwtAuthGuard & RolesGuard • Controller endpoint routing • Domain Service business rules	• Drizzle ORM queries • PostgreSQL ACID transactions • AWS S3 Presigned URLs • CloudFront media delivery

End-to-End Request Lifecycle Specification:

- **1. Client Trigger:** The user initiates an interaction (e.g., submitting the registration form, sending an interest, or searching for matches). React Hook Form validates input against Zod schemas from `@astalakshimi/validation`.
- **2. TanStack Query Execution:** `apps/web/src/hooks/queries.ts` dispatches a request through `apiClient` (`apps/web/src/lib/api-client.ts`), targeting the local Next.js BFF proxy `/api/proxy/\*`.
- **3. BFF Proxy & Cookie Translation:** `apps/web/src/app/api/proxy/[...path]/route.ts` extracts the `astalakshimi.auth\_token` HTTP-only cookie, attaches it as an `Authorization: Bearer` header, and proxies the request to the NestJS backend at `http://localhost:4000/api/\*`.
- **4. Automatic Token Refresh:** If the backend returns a `401 Unauthorized`, the BFF proxy transparently reads `astalakshimi.refresh\_token`, calls `POST /api/auth/refresh`, obtains new tokens, updates the HTTP-only cookies in the response, and retries the original request seamlessly.
- **5. NestJS Guards & Pipes:** In NestJS, `JwtAuthGuard` validates the JWT signature and extracts user context (`req.user = { id, phone, role }`). `ZodValidationPipe` validates the request payload, and `RolesGuard` verifies role permissions.
- **6. Service & ORM Execution:** The controller delegates to domain services (e.g., `ProfilesService`, `InterestsService`), which execute ACID transactions or queries against PostgreSQL via Drizzle ORM (`@astalakshimi/database`).
- **7. External Services Integration:** For media uploads, `S3Provider` generates 10-minute presigned `PUT` URLs. For payments, `PaymentsService` calls Razorpay SDK and verifies SHA-256 HMAC signatures.
- **8. Response & UI Sync:** The NestJS response passes through the BFF proxy back to TanStack Query, which updates cached query keys (e.g., `[profile]`, `[interests]`, `[chat]`) and triggers reactive UI re-renders.

Dual-Bucket Media & Verification Pipeline:

To satisfy strict privacy requirements, media assets are physically segregated into two AWS S3 buckets:

1. **Public Media Bucket (`astalakshimi-media-dev`):** Stores member profile photos (`profiles/{userId}/photos/{uuid}.{ext}`) and horoscope PDFs (`profiles/{userId}/horoscopes/{uuid}.pdf`). Delivered via CloudFront CDN.
2. **Private Vault Bucket (`astalakshimi-vault-dev`):** Stores sensitive verification documents, including live selfies (`verifications/{userId}/selfie-{uuid}.{ext}`) and government IDs (`verifications/{userId}/govt-id-{uuid}.{ext}`). Direct access is blocked; only admins can view documents via 15-minute presigned URLs (`getAdminSignedViewUrl`).

3. Master Feature Inventory & Feature Matrix

The repository contains 15 core feature domains spanning public onboarding, authenticated matchmaking, communication, privacy controls, and administrative moderation.

Feature Domain	Frontend Entry	API Route & Controller	Database Tables	Status
Phone OTP Auth	`/login`, `/register` `hero-register-card`	`POST /auth/send-otp` `POST /auth/verify-otp` `AuthController`	`users` `otp_attempts`	Complete
5-Step Registration	`app/(auth)/register/page.tsx` `useSaveProfileMutation`	`POST /profiles/complete-registration` `ProfilesController`	`profiles`, `family_details`, `lifestyle_interests`, `horoscopes`, `partner_preferences`, `profile_photos`, `verifications`	Complete (8-Table Tx)
Profile Management	`/profile`, `/profile/edit` `useUpdateProfileMutation`	`GET /profiles/me` `PATCH /profiles/me` `ProfilesController`	`profiles`, `family_details`, `lifestyle_interests`, `horoscopes`, `partner_preferences`	Complete
Member Profile View	`app/profiles/[profileId]/page.tsx` Server Component	`GET /profiles/:id` `POST /profiles/:id/visit` `ProfilesController`	`profiles`, `profile_photos`, `profile_views`, `user_settings`, `interests`	Complete (Dynamic Blur)
Discover & Search	`app/(dashboard)/dashboard/page.tsx` `useSearchQuery`	`GET /search` `SearchController`	`profiles`, `profile_photos`, `user_settings`, `interests`	Complete (Active)
Legacy Search	`app/(dashboard)/search/page.tsx` `useMatchesQuery`	None (`useMatchesQuery` returns `[]`)	None	Broken / Dead (Duplicate)
Top Matches	`/home` `useTopMatchesQuery`	`GET /matches/top` `MatchesController`	`profiles`, `profile_photos`, `user_settings`, `interests`	Underengineered (Limit 4)
Interest Dispatch	`MatchListCard`, `ProfileActionBar` `useSendInterestMutation`	`POST /interests` `InterestsController`	`interests` `notifications`	Complete (Auto-Mutual)
Interest Management	`app/(dashboard)/interests/page.tsx` `useInterestsQuery`	`GET /interests/summary` `GET /interests/received` `GET /interests/sent` `GET /interests/mutual`	`interests` `profiles` `profile_photos`	Complete
Chat & Messaging	`app/(dashboard)/inbox/page.tsx` `app/(dashboard)/inbox/[threadId]`	`GET /chat/threads` `GET /chat/:threadId/messages` `POST /chat/:threadId/messages`	`messages` `profiles` `interests`	Complete (3s Polling)
Media S3 Uploads	`profile/edit`, `step-verify` `apiClient.media.getUploadUrl`	`POST /media/upload-url` `POST /media/confirm-photo` `POST /media/confirm-verification`	`profile_photos` `verifications` `horoscopes`	Complete
Shortlisting	`ProfileActionBar`, `shortlists/page.tsx` `useToggleShortlistMutation`	`GET /shortlists` `POST /shortlists` `DELETE /shortlists/:id`	`shortlists` `profiles`	Complete
Notifications	`app/(dashboard)/notifications/page.tsx` `useNotificationsQuery`	`GET /notifications` `PATCH /notifications/:id/read` `DELETE /notifications/clear-all`	`notifications`	Complete
Privacy & Settings	`app/(dashboard)/settings/page.tsx` `useSettingsQuery`	`GET /users/me/settings` `PATCH /users/me/settings` `SettingsController`	`user_settings`	Complete
Plans & Payments	`app/(dashboard)/plans/page.tsx` `app/(dashboard)/checkout/page.tsx`	`GET /plans` `POST /payments/orders` `POST /payments/verify`	`plans` `payments` `subscriptions`	Partial / Mismatch (Backend 4 / Web 5)
Admin Moderation	No Frontend UI page yet (Admin Portal Unscaffolded)	`GET /admin/stats` `GET /admin/verifications/pending` `PATCH /admin/verifications/:id`	`verifications` `users` `profiles`	Backend Only (Guarded by Roles)

4. Deep Feature-by-Feature Technical Workflows

Below is the exact, concrete execution trace for each significant feature across the stack.

Feature 1: Passwordless Mobile Phone OTP Authentication & Session Issuance

User Action: User inputs 10-digit mobile number on `/login` and clicks 'Send OTP'.

- Frontend Trigger:** `apps/web/src/app/(auth)/login/page.tsx` → `handleSendOtp()` invokes `apiClient.auth.sendOtp({ phone, consentAccepted: true })`.
- BFF Route:** Calls Next.js proxy route `/api/proxy/auth/send-otp` → proxied to NestJS `POST /api/auth/send-otp`.
- NestJS AuthController:** `apps/api/src/auth/auth.controller.ts:sendOtp()` receives validated DTO via `ZodValidationPipe(sendOtpSchema)`.
- AuthService Logic:** `apps/api/src/auth/auth.service.ts:sendOtp()` generates a 6-digit OTP (or mock OTP `123456` if `MOCK\_OTP\_ENABLED=true`), calculates expiration (`Date.now() + 300s`), and inserts a record into PostgreSQL table `otp\_attempts`.
- Verification Action:** User inputs 6-digit OTP → `handleVerifyOtp()` invokes `apiClient.auth.verifyOtp()`.
- Cookie Setting:** `apiClient.auth.verifyOtp` calls Next.js route handler `POST /api/auth/login`. This route handler forwards to NestJS `POST /api/auth/verify-otp`. Upon receiving `{ accessToken, refreshToken, user, hasProfile }`, the Next.js handler intercepts the response, sets HTTP-only cookies `astalakshimi.auth\_token` (30 days) and `astalakshimi.refresh\_token` (7 days), strips tokens from the JSON body, and returns `{ user, hasProfile }` to the client.
- Client State Update:** `apiClient.setToken()` sets a flag in localStorage (`is\_authenticated=true`) for synchronous React rendering checks; UI redirects to `/home` (if `hasProfile=true`) or `/register` (if new user).

Feature 2: Multi-Step Registration & 8-Table Atomic Database Transaction

User Action: User completes 5 registration steps on `/register` (Basics → Career → Family → Photos/Verification/Horoscope → Partner Preferences) and submits.

- Frontend Trigger:** `apps/web/src/app/(auth)/register/page.tsx` → calls `saveProfileMutation.mutate(finalData)` (`useSaveProfileMutation` in `apps/web/src/hooks/queries.ts`).
- Payload Construction:** Assembles `CompleteRegistrationPayload` encompassing personal details, DOB, career, income, family values, S3 keys, and preferences.
- API Execution:** Dispatches `POST /api/proxy/profiles/complete-registration` → NestJS `ProfilesController.completeRegistration()`.
- Atomic Transaction Boundary:** `apps/api/src/profiles/profiles.service.ts:completeRegistration()` executes `this.db.transaction(async (tx) => { ... })` across 8 tables:
 - `profiles`: Upserts full name, DOB (`YYYY-MM-DD`), gender, height, education, annual income, photo privacy.
 - `family\_details`: Upserts family values, type, father/mother occupation, siblings count (`onConflictDoUpdate`).
 - `lifestyle\_interests`: Upserts diet, smoking, alcohol, and hobby arrays (`onConflictDoUpdate`).
 - `horoscopes`: Upserts birth time, birth place, manglik status, rasi, nakshatra, and horoscope S3 PDF key.
 - `partner\_preferences`: Upserts age range (min/max), height range, preferred religions, castes, mother tongues.
 - `profile\_photos`: Deletes previous photos for profile, bulk-inserts new photo records with `isPrimary` and `displayOrder`.
 - `verifications`: Upserts verification record with method (`selfie` or `govt\_id`), S3 keys, and status set to `pending`.
- Response & State Update:** Returns `{ success: true, profileId }`. React Query invalidates and updates `queryKeys.profile` cache; UI redirects to `/home`.

Feature 3: Interest Dispatch, Quota Check & Automated Mutual Matching

User Action: Member clicks 'Connect' on a match card (`MatchListCard` or `ProfileActionBar`).

- Frontend Trigger:** `useSendInterestMutation` (`apps/web/src/hooks/queries.ts`) calls `apiClient.interests.sendInterest(targetProfileId, message)`.
- API Routing:** `POST /api/proxy/interests` → NestJS `InterestsController.sendInterest()` (`apps/api/src/interests/interests.controller.ts`).
- Quota Enforcement:** `InterestsService.sendInterest()` checks user's active plan via `EntitlementsService.getUserPlan(userId)`. Queries count of sent interests; if `sentCount >= plan.interestQuota`, throws `403 Forbidden` ('Interest quota limit reached').
- Reverse-Interest Detection (Auto-Match):** Queries `interests` table to check if target user already sent a pending interest to current user. If found:
 - Updates reverse interest status to `accepted` with `respondedAt = now()`.
 - Dispatches asynchronous notifications to both users via `NotificationsService.createNotification()` ('Interest Accepted! You are now connected.').
 - Returns `{ status: 'accepted', isMutual: true }`.
- New Interest Insertion:** If no reverse interest exists, inserts a new record into `interests` with `status: 'pending'` and notifies target user.
- UI Response:** TanStack Query invalidates `queryKeys.interests` and `queryKeys.notifications`. The UI immediately reflects 'Connected' (if mutual) or 'Interest Sent' (if pending).

Feature 4: Real-Time Messaging & Dynamic Thread Resolution

User Action: Member navigates to `/inbox` or `/inbox/[threadId]` and types a message.

1. Active Thread Loading: `useChatMessagesQuery(threadId)` (`apps/web/src/hooks/queries.ts`) executes with `refetchInterval: 3000` (polling every 3 seconds) calling `GET /api/proxy/chat/:threadId/messages`.

2. Backend Resolution: `ChatService.getMessages(userId, threadId)` resolves the partner profile ID, selects messages between sender and receiver ordered by `createdAt ASC`, and executes a background SQL update to mark unread incoming messages as `isRead = true`.

3. Sending Message: User clicks 'Send' → `useSendMessageMutation` performs an optimistic UI update (appending temporary message object), then calls `POST /api/proxy/chat/:threadId/messages`.

4. Persistence & Notification: `ChatService.sendMessage()` computes `threadId` (sorted pair `profileA_profileB` or interest ID), inserts into PostgreSQL `messages` table, and triggers a notification for the recipient.

5. Invalidation: Upon completion, TanStack Query invalidates `queryKeys.chat(threadId)` and `queryKeys.chatThreads`.

5. Master API Inventory Table

Complete catalog of all 37 backend REST endpoints implemented across the 18 NestJS controllers in `apps/api/src`. Every route is mapped with its HTTP method, endpoint path, controller, service method, database tables touched, auth guard, and frontend caller status.

Method & Endpoint	Controller & Service Method	Guards / Auth	Database Tables	Frontend Caller & Status
POST `/auth/send-otp`	AuthController `sendOtp()`	Public ZodPipe	`otp_attempts`	`apiClient.auth.sendOtp` Active
POST `/auth/verify-otp`	AuthController `verifyOtp()`	Public ZodPipe	`otp_attempts` `users`, `profiles`	`app/api/auth/login` Active
POST `/auth/refresh`	AuthController `refresh()`	Public	`users` `profiles`	`app/api/proxy/[...path]` Active (BFF)
GET `/auth/me`	AuthController `getMe()`	JwtAuthGuard	`users` `profiles`	`apiClient.auth.getMe` Active
POST `/profiles/complete-registration`	ProfilesController `completeRegistration()`	JwtAuthGuard ZodPipe	8 Tables (Atomic Tx)	`apiClient.profiles.completeRegistration` Active
GET `/profiles/me`	ProfilesController `getMyProfile()`	JwtAuthGuard	`profiles`, `family_details`, `lifestyle_interests`, etc.	`apiClient.profiles.getMyProfile` Active
PATCH `/profiles/me`	ProfilesController `updateMyProfile()`	JwtAuthGuard	`profiles`, `family_details`, `lifestyle_interests`, etc.	`apiClient.profiles.updateMyProfile` Active
POST `/profiles/me/photos`	ProfilesController `addPhoto()`	JwtAuthGuard	`profile_photos`	`apiClient.photos.add` Active
DELETE `/profiles/me/photos/:photoid`	ProfilesController `deletePhoto()`	JwtAuthGuard	`profile_photos`	`apiClient.photos.remove` Active
PUT `/profiles/me/photos/order`	ProfilesController `reorderPhotos()`	JwtAuthGuard	`profile_photos`	`apiClient.photos.reorder` Active
GET `/profiles/:id`	ProfilesController `getProfileById()`	OptionalJwtGuard	`profiles`, `profile_views`, `user_settings`, `interests`	`app/profiles/[id]/page.tsx` Active
POST `/profiles/:id/visit`	ProfilesController `recordVisit()`	JwtAuthGuard	`profile_views`	`apiClient.profiles.recordVisit` Active
GET `/search`	SearchController `searchProfiles()`	JwtAuthGuard	`profiles`, `profile_photos`, `user_settings`, `interests`	`apiClient.search.searchProfiles` Active
GET `/matches/top`	MatchesController `getTopMatches()`	JwtAuthGuard	`profiles`, `profile_photos`, `user_settings`, `interests`	`apiClient.matches.getTop` Active
POST `/interests`	InterestsController `sendInterest()`	JwtAuthGuard	`interests` `notifications`	`apiClient.interests.sendInterest` Active
GET `/interests/summary`	InterestsController `getSummary()`	JwtAuthGuard	`interests` `profiles`	`apiClient.interests.getSummary` Active
GET `/interests/received`	InterestsController `getReceivedInterests()`	JwtAuthGuard	`interests` `profiles`	`apiClient.interests.getReceived` Active
GET `/interests/sent`	InterestsController `getSentInterests()`	JwtAuthGuard	`interests` `profiles`	`apiClient.interests.getSent` Active
GET `/interests/mutual`	InterestsController `getMutualInterests()`	JwtAuthGuard	`interests` `profiles`	`apiClient.interests.getMutual` Active
PATCH `/interests/:id/accept`	InterestsController `patchAccept()`	JwtAuthGuard	`interests` `notifications`	`apiClient.interests.accept` Active
PATCH `/interests/:id/decline`	InterestsController `patchDecline()`	JwtAuthGuard	`interests`	`apiClient.interests.decline` Active
PATCH `/interests/:id/withdraw`	InterestsController `patchWithdraw()`	JwtAuthGuard	`interests`	`apiClient.interests.withdraw` Active
PUT `/interests/:id/status`	InterestsController `updateInterestStatus()`	JwtAuthGuard	`interests`	`apiClient.interests.updateStatus` Redundant Route
GET `/chat/threads`	ChatController `getThreads()`	JwtAuthGuard	`interests`, `messages`, `profiles`, `profile_photos`	`apiClient.chat.getThreads` Active

GET `/chat/:threadId/messages`	ChatController `getMessages()`	JwtAuthGuard	`messages` `profiles`	`apiClient.chat.getMessages` Active (Poll 3s)
POST `/chat/:threadId/messages`	ChatController `sendMessage()`	JwtAuthGuard	`messages` `notifications`	`apiClient.chat.sendMessage` Active
PATCH `/chat/:threadId/read`	ChatController `markRead()`	JwtAuthGuard	`messages`	`apiClient.chat.markRead` Active
POST `/media/upload-url`	MediaController `getUploadUrl()`	JwtAuthGuard ZodPipe	None (AWS S3 Presigner)	`apiClient.media.getUploadUrl` Active
POST `/media/confirm-photo`	MediaController `confirmPhoto()`	JwtAuthGuard	`profile_photos`	`apiClient.media` / Alt endpoint Redundant Route
POST `/media/confirm-verification`	MediaController `confirmVerification()`	JwtAuthGuard	`verifications`	`apiClient.media` Active
POST `/media/confirm-horoscope`	MediaController `confirmHoroscope()`	JwtAuthGuard	`horoscopes`	`apiClient.media` Active
DELETE `/media/photos/:id`	MediaController `deletePhoto()`	JwtAuthGuard	`profile_photos` + S3 delete	Duplicate of `/profiles/me/photos/:id` Redundant Route
GET `/shortlists`	ShortlistsController `getShortlists()`	JwtAuthGuard	`shortlists` `profiles`	`apiClient.shortlists.getAll` Active
GET `/shortlists/ids`	ShortlistsController `getShortlistIds()`	JwtAuthGuard	`shortlists`	`apiClient.shortlists.getIds` Active
POST `/shortlists`	ShortlistsController `addShortlist()`	JwtAuthGuard	`shortlists`	`apiClient.shortlists.add` Active
DELETE `/shortlists/:targetProfileId`	ShortlistsController `removeShortlist()`	JwtAuthGuard	`shortlists`	`apiClient.shortlists.remove` Active
GET `/notifications`	NotificationsController `getUserNotifications()`	JwtAuthGuard	`notifications`	`apiClient.notifications.getAll` Active
PATCH `/notifications/:id/read`	NotificationsController `markAsRead()`	JwtAuthGuard	`notifications`	`apiClient.notifications.markRead` Active
PATCH `/notifications/read`	NotificationsController `patchMarkRead()`	JwtAuthGuard	`notifications`	`apiClient.notifications.markAllRead` Active
DELETE `/notifications/clear-all`	NotificationsController `clearAll()`	JwtAuthGuard	`notifications`	`apiClient.notifications.clearAll` Active
GET `/users/me/settings`	SettingsController `getSettings()`	JwtAuthGuard	`user_settings`	`apiClient.settings.getSettings` Active
PATCH `/users/me/settings`	SettingsController `updateSettings()`	JwtAuthGuard	`user_settings`	`apiClient.settings.updateSettings` Active
GET `/activity/summary`	ActivityController `getSummary()`	JwtAuthGuard	`profile_views`, `interests`, `shortlists`	`apiClient.activity.getSummary` Active
GET `/plans`	PlansController `getActivePlans()`	Public	`plans`	No Frontend Caller Uncalled API
POST `/payments/orders`	PaymentsController `createOrder()`	JwtAuthGuard	`payments`, `plans`, `profiles`	No Frontend Caller Uncalled API
POST `/payments/verify`	PaymentsController `verifyPayment()`	JwtAuthGuard	`payments`, `subscriptions`, `plans`	No Frontend Caller Uncalled API
GET `/admin/stats`	AdminController `getStats()`	JwtAuthGuard Roles(admin)	`users`, `profiles`, `subscriptions`, `verifications`	Backend Only No UI Page
GET `/admin/verifications/pending`	AdminController `getPendingVerifications()`	JwtAuthGuard Roles(admin)	`verifications`	Backend Only No UI Page
PATCH `/admin/verifications/:profileId`	AdminController `updateVerificationStatus()`	JwtAuthGuard Roles(admin)	`verifications`	Backend Only No UI Page

6. Database Architecture & Schema Specifications

The Astalakshimi database layer consists of 18 PostgreSQL tables declared in `@astalakshimi/database`` using Drizzle ORM. Every table uses UUID primary keys (`defaultRandom()`), strict foreign key cascade rules, timezone-aware timestamps, and enumerated domain types.

Table Name	Primary Key / Foreign Keys	Key Columns & Enums	Indexes & Constraints
<code>`users`</code>	<code>`id`</code> (UUID PK)	<code>`phone`</code> (VARCHAR 15), <code>`is_phone_verified`</code> (BOOL), <code>`consent_accepted`</code> (BOOL), <code>`role`</code> (enum: member, admin, moderator), <code>`status`</code> (enum: active, suspended)	UNIQUE (<code>`phone`</code>)
<code>`profiles`</code>	<code>`id`</code> (UUID PK) <code>`user_id`</code> (FK → users.id ON DELETE CASCADE)	<code>`profile_code`</code> (VARCHAR 20), <code>`full_name`</code> , <code>`gender`</code> (enum), <code>`dob`</code> (DATE), <code>`height_cm`</code> (INT), <code>`marital_status`</code> (enum), <code>`city`</code> , <code>`caste`</code> , <code>`education_level`</code> (enum), <code>`annual_income`</code> , <code>`photo_privacy`</code> (enum)	UNIQUE (<code>`user_id`</code>) UNIQUE (<code>`profile_code`</code>) INDEX (<code>`gender`</code> , <code>`religion`</code> , <code>`caste`</code> , <code>`city`</code>) INDEX (<code>`dob`</code>)
<code>`profile_photos`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE)	<code>`s3_key`</code> (VARCHAR 500), <code>`is_primary`</code> (BOOL), <code>`display_order`</code> (INT), <code>`status`</code> (enum: pending, approved, rejected)	INDEX (<code>`profile_id`</code> , <code>`is_primary`</code>)
<code>`family_details`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE)	<code>`family_values`</code> (enum: Traditional, Moderate, Liberal), <code>`family_type`</code> (enum: Nuclear, Joint), <code>`father_occupation`</code> , <code>`mother_occupation`</code> , <code>`brothers_count`</code> , <code>`sisters_count`</code>	UNIQUE (<code>`profile_id`</code>)
<code>`lifestyle_interests`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE)	<code>`diet`</code> (enum: Vegetarian, Non-vegetarian, Eggetarian, Jain, Vegan), <code>`smoking`</code> (enum), <code>`alcohol`</code> (enum), <code>`interests`</code> (JSONB)	UNIQUE (<code>`profile_id`</code>)
<code>`horoscopes`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE)	<code>`birth_time`</code> , <code>`birth_place`</code> , <code>`manglik`</code> (enum: Yes, No, Don't Know), <code>`rashi`</code> , <code>`nakshatra`</code> , <code>`horoscope_s3_key`</code> (VARCHAR 500), <code>`file_name`</code>	UNIQUE (<code>`profile_id`</code>)
<code>`partner_preferences`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE)	<code>`pref_age_min`</code> , <code>`pref_age_max`</code> , <code>`pref_height_min_cm`</code> , <code>`pref_height_max_cm`</code> , <code>`pref_marital_statuses`</code> (JSONB), <code>`pref_religions`</code> (JSONB), <code>`pref_castes`</code> (JSONB), <code>`pref_mother_tongues`</code> (JSONB)	UNIQUE (<code>`profile_id`</code>)
<code>`verifications`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE)	<code>`method`</code> (enum: selfie, govt_id), <code>`selfie_s3_key`</code> , <code>`govt_id_type`</code> (enum: Aadhaar, PAN card, Passport, Driving licence, Voter ID), <code>`govt_id_s3_key`</code> , <code>`status`</code> (enum: idle, pending, verified, rejected)	UNIQUE (<code>`profile_id`</code>)
<code>`interests`</code>	<code>`id`</code> (UUID PK) <code>`sender_profile_id`</code> (FK → profiles.id) <code>`receiver_profile_id`</code> (FK → profiles.id)	<code>`status`</code> (enum: pending, accepted, declined, withdrawn), <code>`message`</code> (TEXT), <code>`responded_at`</code> (TIMESTAMPTZ)	UNIQUE (<code>`sender_profile_id`</code> , <code>`receiver_profile_id`</code>)
<code>`messages`</code>	<code>`id`</code> (UUID PK) <code>`sender_profile_id`</code> (FK → profiles.id) <code>`receiver_profile_id`</code> (FK → profiles.id)	<code>`thread_id`</code> (VARCHAR 255), <code>`text`</code> (TEXT), <code>`is_read`</code> (BOOL DEFAULT false), <code>`created_at`</code> (TIMESTAMPTZ)	INDEX (<code>`thread_id`</code>) INDEX (<code>`sender_profile_id`</code> , <code>`receiver_profile_id`</code>)
<code>`shortlists`</code>	<code>`id`</code> (UUID PK) <code>`profile_id`</code> (FK → profiles.id CASCADE) <code>`target_profile_id`</code> (FK → profiles.id CASCADE)	<code>`created_at`</code> (TIMESTAMPTZ)	UNIQUE (<code>`profile_id`</code> , <code>`target_profile_id`</code>)
<code>`notifications`</code>	<code>`id`</code> (UUID PK) <code>`user_id`</code> (FK → users.id CASCADE) <code>`actor_profile_id`</code> (FK → profiles.id)	<code>`title`</code> (VARCHAR 150), <code>`body`</code> (TEXT), <code>`category`</code> (VARCHAR 50), <code>`kind`</code> (VARCHAR 50), <code>`is_read`</code> (BOOL), <code>`href`</code> (TEXT)	INDEX (<code>`user_id`</code> , <code>`is_read`</code>)
<code>`plans`</code>	<code>`id`</code> (UUID PK)	<code>`slug`</code> (VARCHAR 20: free, silver, gold, diamond), <code>`name`</code> , <code>`price_paise`</code> (INT), <code>`duration_days`</code> (INT), <code>`interest_quota`</code> (INT), <code>`contact_unlocks`</code> (INT), <code>`has_advanced_filters`</code> (BOOL)	UNIQUE (<code>`slug`</code>)
<code>`payments`</code>	<code>`id`</code> (UUID PK) <code>`user_id`</code> (FK → users.id CASCADE) <code>`plan_id`</code> (FK → plans.id CASCADE)	<code>`amount_paise`</code> (INT), <code>`currency`</code> (INR), <code>`provider`</code> (enum: razorpay, phonepe), <code>`provider_order_id`</code> , <code>`provider_payment_id`</code> , <code>`status`</code> (enum: created, captured, failed, refunded)	UNIQUE (<code>`provider_order_id`</code>) UNIQUE (<code>`provider_payment_id`</code>)

<code>`subscriptions`</code>	<code>`id`</code> (UUID PK) <code>`user_id`</code> (FK → users.id CASCADE) <code>`plan_id`</code> (FK → plans.id CASCADE) <code>`payment_id`</code> (FK → payments.id)	<code>`status`</code> (enum: active, expired, cancelled), <code>`starts_at`</code> (TIMESTAMPTZ), <code>`expires_at`</code> (TIMESTAMPTZ)	INDEX (<code>`user_id`</code> , <code>`status`</code>)
<code>`user_settings`</code>	<code>`id`</code> (UUID PK) <code>`user_id`</code> (FK → users.id CASCADE)	<code>`profile_visibility`</code> (VARCHAR 20), <code>`photo_blur`</code> (VARCHAR 20: always, accepted, never), <code>`hide_phone`</code> (BOOL), <code>`show_last_seen`</code> (BOOL), <code>`hide_from_users`</code> (JSONB), <code>`hide_from_cities`</code> (JSONB)	UNIQUE (<code>`user_id`</code>)
<code>`profile_views`</code>	<code>`id`</code> (UUID PK) <code>`viewer_profile_id`</code> (FK → profiles.id) <code>`target_profile_id`</code> (FK → profiles.id)	<code>`created_at`</code> (TIMESTAMPTZ)	UNIQUE (<code>`viewer_profile_id`</code> , <code>`target_profile_id`</code>)
<code>`otp_attempts`</code>	<code>`id`</code> (UUID PK)	<code>`phone`</code> (VARCHAR 15), <code>`otp_hash`</code> (VARCHAR 255), <code>`expires_at`</code> (TIMESTAMPTZ), <code>`verified`</code> (BOOL), <code>`attempts`</code> (INT)	INDEX (<code>`phone`</code> , <code>`created_at`</code>)

7. Authentication, Authorization & Security Audit

A rigorous security review was conducted across the codebase, identifying confirmed vulnerabilities, architectural security strengths, and missing security controls.

Architectural Security Strengths Implemented:

- **BFF Cookie Protection:** JWT tokens are stored exclusively in HTTP-only, Secure, SameSite=Strict cookies (``astalakshimi.auth_token``), preventing XSS token theft from browser localStorage.
- **Dual-Bucket Segregation:** Sensitive identity documents (Aadhaar, Passport, live selfies) are isolated in a private S3 vault with short-lived (15 min) admin-only presigned URLs.
- **Payment HMAC Verification:** Razorpay webhook and client payments are verified using server-side SHA-256 HMAC cryptographic signature checks (``crypto.createHmac('sha256', secret)``).

Security Findings & Vulnerability Review:

Vulnerability / Finding	Severity & Status	Evidence in Codebase	Remediation Strategy
Plaintext OTP Storage	P0 — Critical Confirmed Issue	<code>`apps/api/src/auth/auth.service.ts:31`</code> <code>`// Save to PostgreSQL (TODO: add proper hashing for otpHash)`</code> Inserts raw OTP digits into <code>`otp_attempts.otpHash`</code> .	Hash OTP with <code>`bcrypt`</code> or <code>`argon2`</code> before inserting into PostgreSQL. Verify using <code>`bcrypt.compare()`</code> .
Unwired Entitlement Guard	P1 — High Confirmed Issue	<code>`apps/api/src/entitlements/entitlement.guard.ts`</code> <code>`@RequireEntitlement()`</code> decorator created but never attached to <code>`/search`</code> or <code>`/matches`</code> controllers.	Attach <code>`@UseGuards(EntitlementGuard)`</code> and <code>`@RequireEntitlement('advanced_filters')`</code> to premium routes.
Direct Schema Creation on Startup	P1 — High Confirmed Issue	<code>`apps/api/src/chat/chat.service.ts:21-34`</code> <code>`onModuleInit()`</code> runs raw SQL <code>`CREATE TABLE IF NOT EXISTS`</code> messages ... on application startup.	Remove raw SQL from service startup lifecycle; manage all tables exclusively via Drizzle migrations.
Unthrottled SMS / OTP Dispatch	P1 — High Potential Risk	<code>`POST /auth/send-otp`</code> lacks IP and phone-number rate limiting guards.	Integrate <code>`@nestjs/throttler`</code> with Redis backend to enforce max 3 OTP requests per phone number per 10 minutes.
IDOR Exposure on Verification Docs	P2 — Medium Architectural Risk	<code>`apps/api/src/admin/admin.controller.ts`</code> <code>`PATCH /admin/verifications/:profileId`</code> relies on <code>`RolesGuard`</code> but lacks audit logging of which admin reviewed the document.	Populate <code>`verifications.reviewedBy = req.user.id`</code> and <code>`reviewedAt = new Date()`</code> on approval/rejection.
Mock Storage Remnants in Web	P3 — Low Technical Debt	<code>`apps/web/src/lib/profile-store.ts`</code> Still writes serialized profile data to <code>`sessionStorage`</code> alongside server DB sync.	Deprecate <code>`sessionStorage`</code> caching; rely purely on TanStack Query cache backed by the REST API.

8. Redundant, Dead & Unused Code Audit

A systematic, file-by-file dead code audit was executed across the entire repository. Every candidate is classified by certainty level (High, Medium, Low) and supported by concrete codebase evidence.

File / Component Path	Category	Evidence & Codebase Analysis	Confidence	Action Recommendation
<code>`apps/web/src/app/(dashboard)/search/page.tsx`</code>	Duplicate Page	Broken duplicate of <code>`/dashboard`</code> . Calls <code>`useMatchesQuery()`</code> which returns <code>[]</code> . All active search is handled by <code>`/dashboard`</code> via <code>`useSearchQuery`</code> .	High	Delete Safely or redirect <code>`/search`</code> to <code>`/dashboard`</code> .
<code>`apps/web/fix_ts.js`</code>	Scratch Script	Temporary AST migration script used during initial cleanup. Not referenced by any build or test pipeline.	High	Delete Immediately.
<code>`test-db.js`</code> <code>`apps/api/test-db.js`</code> <code>`packages/database/test-db.ts`</code>	Scratch Scripts	One-off database connection debugging scripts containing hardcoded localhost connection strings.	High	Delete Immediately.
<code>`apps/web/package-lock.json`</code>	Redundant Lockfile	303 KB npm lockfile committed in <code>`apps/web`</code> while the monorepo root standard is <code>`pnpm-lock.yaml`</code> .	High	Delete Immediately.
<code>`packages/config`</code>	Empty Package	Contains only <code>`.gitkeep`</code> . Constants (plans, religions, enums) are duplicated in <code>`apps/web/src/lib/plans.ts`</code> and <code>`apps/api/src/plans/plans.service.ts`</code> .	High	Consolidate: Move shared plan constants into this package.
<code>`packages/api-client`</code>	Empty Package	Contains only <code>`.gitkeep`</code> . Spec originally planned a shared API client here, but it was embedded into <code>`apps/web/src/lib/api-client.ts`</code> .	High	Consolidate: Extract <code>`apps/web/src/lib/api-client.ts`</code> into this package.
<code>`apps/api/src/entitlements/entitlement.guard.ts`</code>	Unused Guard	<code>`EntitlementGuard`</code> and <code>`@RequireEntitlement`</code> decorator are declared but never attached to any route in any controller.	High	Wire Up to protect premium discovery filters.
<code>`apps/api/src/media/media.controller.ts`</code> (<code>`confirm-photo`</code> , <code>`deletePhoto`</code>)	Redundant APIs	Duplicate endpoints for <code>`POST /profiles/me/photos`</code> and <code>`DELETE /profiles/me/photos/:photoid`</code> in <code>`ProfilesController`</code> .	High	Consolidate into <code>`ProfilesController`</code> or standard media service.
<code>`apps/web/src/components/layout/require-full-portal.tsx`</code> (<code>`if (!true)`</code>)	Disabled Gate	Line 20 has hardcoded <code>`if (!true)`</code> which disables the profile completeness gate across all dashboard routes.	High	Refactor to re-enable dynamic completeness checking.

Architecture Gap Analysis (Specs vs Implementation Reality):

Architectural Dimension	Intended Specification (<code>`specs/*.md`</code>)	Actual Codebase Reality	Evaluation
API Client Location	Shared package <code>`packages/api-client`</code> imported by web and future mobile app.	Local module <code>`apps/web/src/lib/api-client.ts`</code> ; <code>`packages/api-client`</code> is an empty stub with <code>`.gitkeep`</code> .	Diverged
Shared Constants	Shared package <code>`packages/config`</code> owning enums, limits, and plan tiers.	Scattered across <code>`apps/web/src/lib/plans.ts`</code> and <code>`apps/api/src/plans/plans.service.ts`</code> .	Diverged
Membership Plans	Coherent subscription tiers with unified pricing and duration.	Backend seeds 4 plans (Free, Silver █999, Gold █2499, Diamond █4999). Frontend hardcodes 5 plans (Free, Silver █299, Gold █499, Platinum █899, Diamond █1299).	Conflict
Razorpay Integration	End-to-end checkout with Razorpay modal, webhook verification, and subscription updates.	Backend implements full order creation & verification; web frontend uses simulated demo timeout checkout.	Incomplete UI

9. Dependency & Environment Variable Audits

Package Manifest Audit:

Package Name	Declared Location	Status / Usage Evidence	Recommendation
<code>`ts-morph`</code>	<code>`apps/web/package.json`</code> (devDependencies)	Unused in Production Was added only for <code>`fix_ts.js`</code> scratch script.	Remove dependency after deleting <code>`fix_ts.js`</code> .
<code>`crypto`</code>	<code>`apps/api/package.json`</code> (dependencies)	Redundant Package Node.js built-in module. No external npm package required.	Remove <code>`crypto`</code> from <code>`package.json`</code> .
<code>`@aws-sdk/client-s3`</code>	<code>`apps/api/package.json`</code> (dependencies)	Actively Used <code>`apps/api/src/media/providers/s3.provider.ts`</code>	Keep. Core S3 client.
<code>`@tanstack/react-query`</code>	<code>`apps/web/package.json`</code> (dependencies)	Actively Used <code>`apps/web/src/hooks/queries.ts`</code>	Keep. Core server state manager.
<code>`drizzle-orm`</code> + <code>`postgres`</code>	<code>`packages/database/package.json`</code>	Actively Used Database client and schema definitions.	Keep. Primary ORM layer.

Environment Variable Inventory & Exposure Review:

Variable Name	Defined In	Consumer / Service	Exposure / Purpose	Required
<code>`DATABASE_URL`</code>	<code>`.env`</code>	<code>`packages/database/src/client.ts`</code> <code>`apps/api`</code>	PostgreSQL connection string. Server-only secret.	Yes
<code>`JWT_SECRET`</code>	<code>`.env`</code>	<code>`apps/api/src/config/auth.config.ts`</code>	HMAC key for signing JWT tokens. Server-only secret.	Yes
<code>`JWT_EXPIRES_IN`</code>	<code>`.env`</code>	<code>`apps/api/src/config/auth.config.ts`</code>	Access token expiration (e.g., <code>`30d`</code>).	Yes
<code>`MOCK_OTP_ENABLED`</code>	<code>`.env`</code>	<code>`apps/api/src/auth/auth.service.ts`</code>	Boolean flag enabling mock OTP (<code>`123456`</code>) in dev.	No
<code>`AWS_ACCESS_KEY_ID`</code>	<code>`.env`</code>	<code>`apps/api/src/media/providers/s3.provider.ts`</code>	AWS IAM credentials for S3 operations.	Prod
<code>`AWS_SECRET_ACCESS_KEY`</code>	<code>`.env`</code>	<code>`apps/api/src/media/providers/s3.provider.ts`</code>	AWS IAM secret key. High security sensitivity.	Prod
<code>`AWS_S3_MEDIA_BUCKET`</code>	<code>`.env`</code>	<code>`apps/api/src/config/storage.config.ts`</code>	Public profile photos and horoscopes bucket.	Prod
<code>`AWS_S3_VAULT_BUCKET`</code>	<code>`.env`</code>	<code>`apps/api/src/config/storage.config.ts`</code>	Private KYC document and selfie vault bucket.	Prod
<code>`CLOUDFRONT_URL`</code>	<code>`.env`</code>	<code>`apps/web/src/lib/utils.ts`</code>	CDN domain for serving optimized public photos.	Prod
<code>`NEXT_PUBLIC_API_URL`</code>	<code>`.env`</code>	<code>`apps/web`</code> BFF proxy handlers	Target backend URL for proxy (<code>`http://localhost:4000/api`</code>).	Yes
<code>`RAZORPAY_KEY_ID`</code>	<code>`.env`</code>	<code>`apps/api/src/payments/payments.service.ts`</code>	Razorpay API public key.	Prod
<code>`RAZORPAY_KEY_SECRET`</code>	<code>`.env`</code>	<code>`apps/api/src/payments/payments.service.ts`</code>	Razorpay API secret for signature verification.	Prod

10. Performance Bottlenecks & Priority Action Matrix

Senior engineering evaluation of system bottlenecks under 10× traffic load and prioritized remediation steps.

Priority	Issue & Vulnerability	Impact on Production	Concrete Remediation Action
P0 — Critical	Plaintext OTP storage in PostgreSQL `otp_attempts` table.	Security breach risk: Any DB compromise exposes active login codes.	Hash OTP with `bcrypt`/`argon2` prior to insertion; compare hashes during verification.
P0 — Critical	Raw SQL migration on startup in `ChatService.onModuleInit`.	Race conditions in multi-instance auto-scaling; bypasses Drizzle migration meta journal.	Generate proper Drizzle migration file for `messages` table and remove raw SQL from service.
P1 — High	N+1 DB queries in `ChatService.getThreads`.	Database connection exhaustion at scale; thread listing slows linearly with match count.	Replace `Promise.all` map loop with a single SQL query using `LEFT JOIN` and `COUNT(*) GROUP BY`.
P1 — High	Aggressive 3-second chat polling in `hooks/queries.ts`.	Overwhelming API compute: 1,000 active chat windows generate 20,000 HTTP requests/min.	Migrate chat transport to WebSockets (NestJS Gateway + Socket.io) or Server-Sent Events (SSE).
P1 — High	Memory-based total count in `SearchService.searchProfiles`.	Out-of-memory crashes: Fetches all matching profile IDs into memory to compute `.length`.	Execute a dedicated `SELECT COUNT(*)` query alongside the paginated query.
P2 — Medium	Plan tiers and pricing mismatch (Backend 4 vs Web 5).	Pricing confusion and payment failures during Razorpay order creation.	Align `apps/web/src/lib/plans.ts` with `packages/database/src/schema/plans.ts` or fetch plans dynamically from `GET /plans`.
P3 — Low	Leftover scratch scripts and duplicate npm lockfile (`apps/web/package-lock.json`).	Developer confusion and repository bloat.	Delete `apps/web/fix_ts.js`, `test-db.js`, `apps/api/test-db.js`, and `apps/web/package-lock.json`.

Final Architectural Verdict:

Architectural Coherence: The Astalakshimi repository possesses a strong architectural foundation. The monorepo structure, BFF cookie authentication proxy, Drizzle schema relational design, and S3 dual-bucket vault separation are clean and production-oriented.

Overengineered vs Underengineered Areas:

- *Overengineered:* Redundant controller endpoints (multiple verbs and paths doing identical operations in `InterestsController` and `NotificationsController`).
- *Underengineered:* Match scoring algorithm (hardcoded `limit(4)` without preference ranking) and real-time messaging (polling every 3s instead of WebSockets).

What Breaks at 10× Traffic:

1. **Chat Polling:** 3-second polling against `GET /chat/:threadId/messages` and `GET /chat/threads` will exhaust PostgreSQL connection pools.
2. **Search Total Count:** `SearchService` pulling all IDs into memory to compute `.length` will exhaust Node.js heap memory on 50,000+ member profiles.

Strategic Recommendation: Address P0/P1 security and performance remediations first, delete confirmed dead artifacts (`fix\_ts.js`, `package-lock.json`, `/search` page), and transition chat from HTTP polling to WebSockets before public launch.